

# Chapter 5 - Simple Offline Music Player

---

## Objective

In this exercise, you will create a fully functional offline music player application using Flutter that plays audio files stored locally on the device. This project will help you understand:

- Audio playback management in Flutter
- File system access and local storage
- Media controls and audio state management
- Creating custom audio player UI
- Background audio playback
- Playlist management
- Audio metadata extraction
- Handling audio lifecycle events

## Responsibilities

As a student, you are expected to:

1. **Audio Integration:** Implement audio playback with play, pause, stop, seek functionality
2. **File Management:** Access and manage local audio files from device storage
3. **UI/UX Design:** Create an intuitive and visually appealing music player interface
4. **State Management:** Handle playback state, playlist state, and audio progress
5. **Playlist Management:** Create, edit, and manage playlists
6. **Audio Controls:** Implement shuffle, repeat, previous, next, and seek controls
7. **Metadata Display:** Show song title, artist, album, and artwork
8. **Background Playback:** Continue playing music when app is in background
9. **Persistence:** Save user preferences, playlists, and playback state

## Resources

### Figma Design

- **Design Link:** [Your Music Player Design]
- **Design Specifications:**
  - **Colors:**
    - Primary: #1DB954 (Spotify Green)
    - Secondary: #191414 (Dark Background)
    - Accent: #FFFFFF (White Text)
    - Card Background: #282828
  - **Font:** Circular Std / Roboto
  - **Player height:** 80dp (mini player), Full screen (expanded player)
  - **Button size:** 48dp for main controls, 40dp for secondary controls
  - **Album art:** Square with 8dp border radius
  - **Screen padding:** 16dp

## Sample Audio Files

For testing purposes, you can use:

- **Free Music Archive:** <https://freemusicarchive.org/>
- **YouTube Audio Library:** <https://www.youtube.com/audiolibrary>
- **Bensound:** <https://www.bensound.com/>
- **Incompetech:** <https://incompetech.com/music/>
- **ccMixter:** <http://ccmixter.org/>

**Note:** Ensure all audio files are properly licensed for use.

## Additional Resources

- **just\_audio Package:** [https://pub.dev/packages/just\\_audio](https://pub.dev/packages/just_audio)
- **audio\_service Package:** [https://pub.dev/packages/audio\\_service](https://pub.dev/packages/audio_service)
- **path\_provider Package:** [https://pub.dev/packages/path\\_provider](https://pub.dev/packages/path_provider)
- **permission\_handler Package:** [https://pub.dev/packages/permission\\_handler](https://pub.dev/packages/permission_handler)
- **file\_picker Package:** [https://pub.dev/packages/file\\_picker](https://pub.dev/packages/file_picker)
- **on\_audio\_query Package:** [https://pub.dev/packages/on\\_audio\\_query](https://pub.dev/packages/on_audio_query)
- **audio\_session Package:** [https://pub.dev/packages/audio\\_session](https://pub.dev/packages/audio_session)
- **rxdart Package:** <https://pub.dev/packages/rxdart>

## Instructions

### Step 1: Project Setup

#### 1. Create a new Flutter project:

```
flutter create offline_music_player  
cd offline_music_player
```

#### 2. Add dependencies to `pubspec.yaml`:

```
dependencies:  
  flutter:  
    sdk: flutter  
  just_audio: ^0.9.36  
  audio_service: ^0.18.12  
  provider: ^6.1.1  
  shared_preferences: ^2.2.2  
  path_provider: ^2.1.1  
  permission_handler: ^11.1.0  
  file_picker: ^6.1.1  
  on_audio_query: ^2.9.0  
  audio_session: ^0.1.18  
  rxdart: ^0.27.7  
  palette_generator: ^0.3.3+3  
  cached_network_image: ^3.3.0
```

```
dev_dependencies:  
  flutter_test:  
    sdk: flutter  
  flutter_launcher_icons: ^0.13.1
```

### 3. Configure Android permissions in `android/app/src/main/AndroidManifest.xml`:

```
<manifest>  
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>  
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>  
  <uses-permission android:name="android.permission.READ_MEDIA_AUDIO"/>  
  <uses-permission android:name="android.permission.WAKE_LOCK"/>  
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>  
  
  <application>  
    <!-- Add this for background audio -->  
    <service android:name="com.ryanheise.audioservice.AudioService"  
      android:foregroundServiceType="mediaPlayback"  
      android:exported="true">  
      <intent-filter>  
        <action android:name="android.media.browse.MediaBrowserService"/>  
      </intent-filter>  
    </service>  
  </application>  
</manifest>
```

### 4. Configure iOS permissions in `ios/Runner/Info.plist`:

```
<key>NSAppleMusicUsageDescription</key>  
<string>This app needs access to your music library</string>  
<key>UIBackgroundModes</key>  
<array>  
  <string>audio</string>  
</array>
```

### 5. Run: `flutter pub get`

## Step 2: Project Structure

```
lib/  
├─ main.dart  
├─ models/  
│   ├─ song_model.dart  
│   ├─ playlist_model.dart  
│   └─ playback_state_model.dart  
└─ services/
```

```
|   ├── audio_player_service.dart
|   ├── storage_service.dart
|   ├── permission_service.dart
|   └── playlist_service.dart
|── providers/
|   ├── audio_provider.dart
|   ├── playlist_provider.dart
|   └── theme_provider.dart
|── screens/
|   ├── home_screen.dart
|   ├── now_playing_screen.dart
|   ├── playlist_screen.dart
|   ├── all_songs_screen.dart
|   └── settings_screen.dart
|── widgets/
|   ├── song_tile.dart
|   ├── mini_player.dart
|   ├── player_controls.dart
|   ├── progress_bar.dart
|   ├── playlist_card.dart
|   └── album_art.dart
|── utils/
|   ├── constants.dart
|   ├── duration_formatter.dart
|   └── color_extractor.dart
|
|assets/
| ├── audio/
| │   └── sample_songs/
| ├── images/
| │   └── default_album_art.png
```

## Step 3: Model Classes

### 1. Song Model:

```
// lib/models/song_model.dart
class SongModel {
  final String id;
  final String title;
  final String artist;
  final String? album;
  final String filePath;
  final Duration? duration;
  final String? albumArt;
  final int? fileSize;

  SongModel({
    required this.id,
    required this.title,
    required this.artist,
```

```

        this.album,
        required this.filePath,
        this.duration,
        this.albumArt,
        this.fileSize,
    });

    factory SongModel.fromJson(Map<String, dynamic> json) {
        return SongModel(
            id: json['id'],
            title: json['title'],
            artist: json['artist'],
            album: json['album'],
            filePath: json['filePath'],
            duration: json['duration'] != null
                ? Duration(milliseconds: json['duration'])
                : null,
            albumArt: json['albumArt'],
            fileSize: json['fileSize'],
        );
    }

    Map<String, dynamic> toJson() {
        return {
            'id': id,
            'title': title,
            'artist': artist,
            'album': album,
            'filePath': filePath,
            'duration': duration?.inMilliseconds,
            'albumArt': albumArt,
            'fileSize': fileSize,
        };
    }

    // Create from AudioQuery
    factory SongModel.fromAudioQuery(dynamic audioModel) {
        return SongModel(
            id: audioModel.id.toString(),
            title: audioModel.title,
            artist: audioModel.artist ?? 'Unknown Artist',
            album: audioModel.album,
            filePath: audioModel.data,
            duration: Duration(milliseconds: audioModel.duration ?? 0),
        );
    }
}

```

## 2. Playlist Model:

```
// lib/models/playlist_model.dart
class PlaylistModel {
  final String id;
  final String name;
  final List<String> songIds;
  final DateTime createdAt;
  final DateTime updatedAt;
  final String? coverImage;

  PlaylistModel({
    required this.id,
    required this.name,
    required this.songIds,
    required this.createdAt,
    required this.updatedAt,
    this.coverImage,
  });

  factory PlaylistModel.fromJson(Map<String, dynamic> json) {
    return PlaylistModel(
      id: json['id'],
      name: json['name'],
      songIds: List<String>.from(json['songIds']),
      createdAt: DateTime.parse(json['createdAt']),
      updatedAt: DateTime.parse(json['updatedAt']),
      coverImage: json['coverImage'],
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'name': name,
      'songIds': songIds,
      'createdAt': createdAt.toIso8601String(),
      'updatedAt': updatedAt.toIso8601String(),
      'coverImage': coverImage,
    };
  }

  PlaylistModel copyWith({
    String? name,
    List<String>? songIds,
    DateTime? updatedAt,
    String? coverImage,
  }) {
    return PlaylistModel(
      id: this.id,
      name: name ?? this.name,
      songIds: songIds ?? this.songIds,
      createdAt: this.createdAt,
      updatedAt: updatedAt ?? DateTime.now(),
      coverImage: coverImage ?? this.coverImage,
    );
  }
}
```

```

    );
  }
}

```

## Step 4: Services Implementation

### 1. Audio Player Service:

```

// lib/services/audio_player_service.dart
import 'package:just_audio/just_audio.dart';
import 'package:rxdart/rxdart.dart';

class AudioPlayerService {
  final AudioPlayer _audioPlayer = AudioPlayer();

  // Streams
  Stream<Duration> get positionStream => _audioPlayer.positionStream;
  Stream<Duration?> get durationStream => _audioPlayer.durationStream;
  Stream<PlayerState> get playerStateStream => _audioPlayer.playerStateStream;
  Stream<bool> get playingStream => _audioPlayer.playingStream;

  // Current state getters
  Duration get currentPosition => _audioPlayer.position;
  Duration? get currentDuration => _audioPlayer.duration;
  bool get isPlaying => _audioPlayer.playing;

  // Playback state stream combining position, duration, and playing state
  Stream<PlaybackState> get playbackStateStream {
    return Rx.combineLatest3<Duration, Duration?, bool, PlaybackState>(
      positionStream,
      durationStream,
      playingStream,
      (position, duration, isPlaying) => PlaybackState(
        position: position,
        duration: duration ?? Duration.zero,
        isPlaying: isPlaying,
      ),
    );
  }

  // Load and play audio from file path
  Future<void> loadAudio(String filePath) async {
    try {
      await _audioPlayer.setFilePath(filePath);
    } catch (e) {
      throw Exception('Error loading audio: $e');
    }
  }

  // Play
  Future<void> play() async {

```

```
    await _audioPlayer.play();
}

// Pause
Future<void> pause() async {
    await _audioPlayer.pause();
}

// Stop
Future<void> stop() async {
    await _audioPlayer.stop();
}

// Seek to position
Future<void> seek(Duration position) async {
    await _audioPlayer.seek(position);
}

// Set volume (0.0 to 1.0)
Future<void> setVolume(double volume) async {
    await _audioPlayer.setVolume(volume);
}

// Set speed (0.5 to 2.0)
Future<void> setSpeed(double speed) async {
    await _audioPlayer.setSpeed(speed);
}

// Set loop mode
Future<void> setLoopMode(LoopMode loopMode) async {
    await _audioPlayer.setLoopMode(loopMode);
}

// Dispose
void dispose() {
    _audioPlayer.dispose();
}
}

// Playback state model
class PlaybackState {
    final Duration position;
    final Duration duration;
    final bool isPlaying;

    PlaybackState({
        required this.position,
        required this.duration,
        required this.isPlaying,
    });

    double get progress {
        if (duration.inMilliseconds > 0) {
            return position.inMilliseconds / duration.inMilliseconds;
        }
    }
}
```



```
    }  
    return 0.0;  
  }  
}
```

## 2. Permission Service:

```
// lib/services/permission_service.dart  
import 'package:permission_handler/permission_handler.dart';  
  
class PermissionService {  
  // Request storage permission  
  Future<bool> requestStoragePermission() async {  
    var status = await Permission.storage.status;  
  
    if (status.isGranted) {  
      return true;  
    }  
  
    if (status.isDenied) {  
      status = await Permission.storage.request();  
      return status.isGranted;  
    }  
  
    if (status.isPermanentlyDenied) {  
      await openAppSettings();  
      return false;  
    }  
  
    return false;  
  }  
  
  // Request audio permission (Android 13+)  
  Future<bool> requestAudioPermission() async {  
    if (await Permission.audio.isGranted) {  
      return true;  
    }  
  
    var status = await Permission.audio.request();  
  
    if (status.isPermanentlyDenied) {  
      await openAppSettings();  
      return false;  
    }  
  
    return status.isGranted;  
  }  
  
  // Check if permissions are granted  
  Future<bool> hasPermissions() async {  
    bool storagePermission = await Permission.storage.isGranted;
```

```
    bool audioPermission = await Permission.audio.isGranted;

    return storagePermission || audioPermission;
  }
}
```

### 3. Storage Service:

```
// lib/services/storage_service.dart
import 'dart:convert';
import 'package:shared_preferences/shared_preferences.dart';

class StorageService {
  static const String _playlistsKey = 'playlists';
  static const String _lastPlayedKey = 'last_played';
  static const String _shuffleKey = 'shuffle_enabled';
  static const String _repeatKey = 'repeat_mode';
  static const String _volumeKey = 'volume';

  // Save playlists
  Future<void> savePlaylists(List<PlaylistModel> playlists) async {
    final prefs = await SharedPreferences.getInstance();
    final playlistsJson = playlists.map((p) => p.toJson()).toList();
    await prefs.setString(_playlistsKey, json.encode(playlistsJson));
  }

  // Get playlists
  Future<List<PlaylistModel>> getPlaylists() async {
    final prefs = await SharedPreferences.getInstance();
    final playlistsString = prefs.getString(_playlistsKey);

    if (playlistsString != null) {
      final List<dynamic> playlistsJson = json.decode(playlistsString);
      return playlistsJson
        .map((json) => PlaylistModel.fromJson(json))
        .toList();
    }

    return [];
  }

  // Save last played song
  Future<void> saveLastPlayed(String songId) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_lastPlayedKey, songId);
  }

  // Get last played song
  Future<String?> getLastPlayed() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_lastPlayedKey);
  }
}
```

```

    }

    // Save shuffle state
    Future<void> saveShuffleState(bool enabled) async {
      final prefs = await SharedPreferences.getInstance();
      await prefs.setBool(_shuffleKey, enabled);
    }

    // Get shuffle state
    Future<bool> getShuffleState() async {
      final prefs = await SharedPreferences.getInstance();
      return prefs.getBool(_shuffleKey) ?? false;
    }

    // Save repeat mode (0: off, 1: all, 2: one)
    Future<void> saveRepeatMode(int mode) async {
      final prefs = await SharedPreferences.getInstance();
      await prefs.setInt(_repeatKey, mode);
    }

    // Get repeat mode
    Future<int> getRepeatMode() async {
      final prefs = await SharedPreferences.getInstance();
      return prefs.getInt(_repeatKey) ?? 0;
    }

    // Save volume
    Future<void> saveVolume(double volume) async {
      final prefs = await SharedPreferences.getInstance();
      await prefs.setDouble(_volumeKey, volume);
    }

    // Get volume
    Future<double> getVolume() async {
      final prefs = await SharedPreferences.getInstance();
      return prefs.getDouble(_volumeKey) ?? 1.0;
    }
  }
}

```

#### 4. Playlist Service:

```

// lib/services/playlist_service.dart
import 'package:on_audio_query/on_audio_query.dart';

class PlaylistService {
  final OnAudioQuery _audioQuery = OnAudioQuery();

  // Get all songs from device
  Future<List<SongModel>> getAllSongs() async {
    try {
      final List<SongModel> audioList = await _audioQuery.querySongs(

```

```

        sortType: SongSortType.TITLE,
        orderType: OrderType.ASC_OR_SMALLER,
        uriType: UriType.EXTERNAL,
        ignoreCase: true,
    );

    return audioList.map((audio) => SongModel.fromAudioQuery(audio)).toList();
  } catch (e) {
    throw Exception('Error loading songs: $e');
  }
}

// Get songs by artist
Future<List<SongModel>> getSongsByArtist(String artist) async {
  final allSongs = await getAllSongs();
  return allSongs.where((song) => song.artist == artist).toList();
}

// Get songs by album
Future<List<SongModel>> getSongsByAlbum(String album) async {
  final allSongs = await getAllSongs();
  return allSongs.where((song) => song.album == album).toList();
}

// Search songs
Future<List<SongModel>> searchSongs(String query) async {
  final allSongs = await getAllSongs();
  final lowerQuery = query.toLowerCase();

  return allSongs.where((song) {
    return song.title.toLowerCase().contains(lowerQuery) ||
           song.artist.toLowerCase().contains(lowerQuery) ||
           (song.album?.toLowerCase().contains(lowerQuery) ?? false);
  }).toList();
}
}

```

## Step 5: Provider Implementation

```

// lib/providers/audio_provider.dart
import 'package:flutter/material.dart';
import 'package:just_audio/just_audio.dart';

class AudioProvider extends ChangeNotifier {
  final AudioPlayerService _audioService;
  final StorageService _storageService;

  List<SongModel> _playlist = [];
  int _currentIndex = 0;
  bool _isShuffleEnabled = false;
  LoopMode _loopMode = LoopMode.off;
}

```

```
AudioProvider(this._audioService, this._storageService) {
  _init();
}

// Getters
List<SongModel> get playlist => _playlist;
int get currentIndex => _currentIndex;
SongModel? get currentSong => _playlist.isEmpty ? null :
_playlist[_currentIndex];
bool get isShuffleEnabled => _isShuffleEnabled;
LoopMode get loopMode => _loopMode;

Stream<Duration> get positionStream => _audioService.positionStream;
Stream<Duration?> get durationStream => _audioService.durationStream;
Stream<bool> get playingStream => _audioService.playingStream;
Stream<PlaybackState> get playbackStateStream =>
_audioService.playbackStateStream;

// Initialize
Future<void> _init() async {
  _isShuffleEnabled = await _storageService.getShuffleState();
  final repeatMode = await _storageService.getRepeatMode();
  _loopMode = LoopMode.values[repeatMode];
  await _audioService.setLoopMode(_loopMode);

  final volume = await _storageService.getVolume();
  await _audioService.setVolume(volume);
}

// Set playlist
Future<void> setPlaylist(List<SongModel> songs, int startIndex) async {
  _playlist = songs;
  _currentIndex = startIndex;
  await _playSongAtIndex(_currentIndex);
  notifyListeners();
}

// Play song at index
Future<void> _playSongAtIndex(int index) async {
  if (index < 0 || index >= _playlist.length) return;

  _currentIndex = index;
  final song = _playlist[index];

  await _audioService.loadAudio(song.filePath);
  await _audioService.play();
  await _storageService.saveLastPlayed(song.id);

  notifyListeners();
}

// Play/Pause
Future<void> playPause() async {
```

```
    if (_audioService.isPlaying) {
        await _audioService.pause();
    } else {
        await _audioService.play();
    }
    notifyListeners();
}

// Next song
Future<void> next() async {
    if (_isShuffleEnabled) {
        _currentIndex = _getRandomIndex();
    } else {
        _currentIndex = (_currentIndex + 1) % _playlist.length;
    }
    await _playSongAtIndex(_currentIndex);
}

// Previous song
Future<void> previous() async {
    if (_audioService.currentPosition.inSeconds > 3) {
        await _audioService.seek(Duration.zero);
    } else {
        if (_isShuffleEnabled) {
            _currentIndex = _getRandomIndex();
        } else {
            _currentIndex = (_currentIndex - 1 + _playlist.length) % _playlist.length;
        }
        await _playSongAtIndex(_currentIndex);
    }
}

// Seek
Future<void> seek(Duration position) async {
    await _audioService.seek(position);
}

// Toggle shuffle
Future<void> toggleShuffle() async {
    _isShuffleEnabled = !_isShuffleEnabled;
    await _storageService.saveShuffleState(_isShuffleEnabled);
    notifyListeners();
}

// Toggle repeat
Future<void> toggleRepeat() async {
    switch (_loopMode) {
        case LoopMode.off:
            _loopMode = LoopMode.all;
            break;
        case LoopMode.all:
            _loopMode = LoopMode.one;
            break;
        case LoopMode.one:
```

```

        _loopMode = LoopMode.off;
        break;
    }

    await _audioService.setLoopMode(_loopMode);
    await _storageService.saveRepeatMode(_loopMode.index);
    notifyListeners();
}

// Set volume
Future<void> setVolume(double volume) async {
    await _audioService.setVolume(volume);
    await _storageService.saveVolume(volume);
    notifyListeners();
}

// Get random index
int _getRandomIndex() {
    final random = DateTime.now().millisecondsSinceEpoch % _playlist.length;
    return random;
}

@override
void dispose() {
    _audioService.dispose();
    super.dispose();
}
}

```

## Step 6: UI Implementation

### 1. Home Screen:

```

// lib/screens/home_screen.dart
class HomeScreen extends StatefulWidget {
    @override
    _HomeScreenState createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
    final PlaylistService _playlistService = PlaylistService();
    final PermissionService _permissionService = PermissionService();

    List<SongModel> _songs = [];
    bool _isLoading = true;
    bool _hasPermission = false;

    @override
    void initState() {
        super.initState();
        _initializeApp();
    }
}

```

```

}

Future<void> _initializeApp() async {
  // Request permissions
  _hasPermission = await _permissionService.requestStoragePermission();

  if (_hasPermission) {
    // On Android 13+, also request audio permission
    await _permissionService.requestAudioPermission();
    await _loadSongs();
  }

  setState(() {
    _isLoading = false;
  });
}

Future<void> _loadSongs() async {
  try {
    final songs = await _playlistService.getAllSongs();
    setState(() {
      _songs = songs;
    });
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error loading songs: $e')),
    );
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Color(0xFF191414),
    body: SafeArea(
      child: Column(
        children: [
          // App Bar
          _buildAppBar(),

          // Content
          Expanded(
            child: _isLoading
              ? Center(child: CircularProgressIndicator())
              : !_hasPermission
                ? _buildPermissionDenied()
                : _songs.isEmpty
                  ? _buildNoSongs()
                  : _buildSongList(),
          ),

          // Mini Player
          Consumer<AudioProvider>(
            builder: (context, provider, child) {

```



```

        if (provider.currentSong == null) return SizedBox.shrink();
        return MiniPlayer();
      },
    ),
  ],
),
),
);
}

Widget _buildAppBar() {
  return Padding(
    padding: EdgeInsets.all(16),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text(
          'My Music',
          style: TextStyle(
            color: Colors.white,
            fontSize: 28,
            fontWeight: FontWeight.bold,
          ),
        ),
        IconButton(
          icon: Icon(Icons.search, color: Colors.white),
          onPressed: () {
            // Navigate to search screen
          },
        ),
      ],
    ),
  );
}

Widget _buildSongList() {
  return ListView.builder(
    itemCount: _songs.length,
    itemBuilder: (context, index) {
      final song = _songs[index];
      return SongTile(
        song: song,
        onTap: () {
          context.read<AudioProvider>().setPlaylist(_songs, index);
        },
      );
    },
  );
}

Widget _buildPermissionDenied() {
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,

```

```

        children: [
          Icon(Icons.music_off, size: 80, color: Colors.grey),
          SizedBox(height: 20),
          Text(
            'Storage Permission Required',
            style: TextStyle(color: Colors.white, fontSize: 20),
          ),
          SizedBox(height: 10),
          Text(
            'Please grant storage permission to access music',
            style: TextStyle(color: Colors.grey),
            textAlign: TextAlign.center,
          ),
          SizedBox(height: 20),
          ElevatedButton(
            onPressed: () async {
              await openAppSettings();
            },
            child: Text('Open Settings'),
          ),
        ],
      ),
    );
  }

  Widget _buildNoSongs() {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(Icons.music_note, size: 80, color: Colors.grey),
          SizedBox(height: 20),
          Text(
            'No Music Found',
            style: TextStyle(color: Colors.white, fontSize: 20),
          ),
          SizedBox(height: 10),
          Text(
            'Add some music files to your device',
            style: TextStyle(color: Colors.grey),
          ),
        ],
      ),
    );
  }
}

```

## 2. Now Playing Screen:

```

// lib/screens/now_playing_screen.dart
class NowPlayingScreen extends StatelessWidget {

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Color(0xFF191414),
    body: Consumer<AudioProvider>(
      builder: (context, provider, child) {
        final song = provider.currentSong;

        if (song == null) {
          return Center(child: Text('No song playing'));
        }

        return SafeArea(
          child: Column(
            children: [
              // App Bar
              _buildAppBar(context),

              Expanded(
                child: Padding(
                  padding: EdgeInsets.symmetric(horizontal: 24),
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      // Album Art
                      _buildAlbumArt(song),

                      SizedBox(height: 40),

                      // Song Info
                      _buildSongInfo(song),

                      SizedBox(height: 40),

                      // Progress Bar
                      StreamBuilder<PlaybackState>(
                        stream: provider.playbackStateStream,
                        builder: (context, snapshot) {
                          final state = snapshot.data;
                          return ProgressBar(
                            position: state?.position ?? Duration.zero,
                            duration: state?.duration ?? Duration.zero,
                            onSeek: (position) {
                              provider.seek(position);
                            },
                          ),
                        ),
                      ),

                      SizedBox(height: 20),

                      // Player Controls
                      PlayerControls(provider: provider),
                    ],
                  ),
                ),
              ),
            ],
          ),
        );
      },
    ),
  );
}

```

```

        ),
      ),
    ),
  ],
),
);
},
),
);
}

```

```

Widget _buildAppBar(BuildContext context) {
  return Padding(
    padding: EdgeInsets.all(16),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        IconButton(
          icon: Icon(Icons.keyboard_arrow_down, color: Colors.white, size: 32),
          onPressed: () => Navigator.pop(context),
        ),
        Text(
          'Now Playing',
          style: TextStyle(color: Colors.white, fontSize: 16),
        ),
        IconButton(
          icon: Icon(Icons.more_vert, color: Colors.white),
          onPressed: () {
            // Show options menu
          },
        ),
      ],
    ),
  );
}

```

```

Widget _buildAlbumArt(SongModel song) {
  return Container(
    width: 300,
    height: 300,
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(8),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withOpacity(0.5),
          blurRadius: 20,
          offset: Offset(0, 10),
        ),
      ],
    ),
    child: ClipRRect(
      borderRadius: BorderRadius.circular(8),
      child: song.albumArt != null
        ? Image.file(File(song.albumArt!), fit: BoxFit.cover)

```

```

        : Container(
          color: Color(0xFF282828),
          child: Icon(Icons.music_note, size: 100, color: Colors.grey),
        ),
      ),
    );
  }

Widget _buildSongInfo(SongModel song) {
  return Column(
    children: [
      Text(
        song.title,
        style: TextStyle(
          color: Colors.white,
          fontSize: 24,
          fontWeight: FontWeight.bold,
        ),
        textAlign: TextAlign.center,
        maxLines: 2,
        overflow: TextOverflow.ellipsis,
      ),
      SizedBox(height: 8),
      Text(
        song.artist,
        style: TextStyle(
          color: Colors.grey,
          fontSize: 16,
        ),
        textAlign: TextAlign.center,
      ),
    ],
  );
}

```

### 3. Song Tile Widget:

```

// lib/widgets/song_tile.dart
class SongTile extends StatelessWidget {
  final SongModel song;
  final VoidCallback onTap;

  const SongTile({
    required this.song,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return ListTile(

```

```

        contentPadding: EdgeInsets.symmetric(horizontal: 16, vertical: 8),
        leading: _buildAlbumArt(),
        title: Text(
          song.title,
          style: TextStyle(color: Colors.white, fontWeight: FontWeight.w500),
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
        ),
        subtitle: Text(
          song.artist,
          style: TextStyle(color: Colors.grey),
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
        ),
        trailing: IconButton(
          icon: Icon(Icons.more_vert, color: Colors.grey),
          onPressed: () {
            _showOptionsMenu(context);
          },
        ),
        onTap: onTap,
      );
    }

    Widget _buildAlbumArt() {
      return Container(
        width: 50,
        height: 50,
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(4),
          color: Color(0xFF282828),
        ),
        child: song.albumArt != null
          ? ClipRRect(
              borderRadius: BorderRadius.circular(4),
              child: Image.file(File(song.albumArt!), fit: BoxFit.cover),
            )
          : Icon(Icons.music_note, color: Colors.grey),
      );
    }

    void _showOptionsMenu(BuildContext context) {
      showModalBottomSheet(
        context: context,
        backgroundColor: Color(0xFF282828),
        builder: (context) {
          return Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              ListTile(
                leading: Icon(Icons.playlist_add, color: Colors.white),
                title: Text('Add to playlist', style: TextStyle(color:
Colors.white)),
                onTap: () {

```

```

        // Add to playlist
        Navigator.pop(context);
      },
    ),
    ListTile(
      leading: Icon(Icons.share, color: Colors.white),
      title: Text('Share', style: TextStyle(color: Colors.white)),
      onTap: () {
        // Share song
        Navigator.pop(context);
      },
    ),
    ListTile(
      leading: Icon(Icons.info_outline, color: Colors.white),
      title: Text('Song info', style: TextStyle(color: Colors.white)),
      onTap: () {
        // Show song info
        Navigator.pop(context);
      },
    ),
  ],
);
},
);
}
}

```

#### 4. Mini Player Widget:

```

// lib/widgets/mini_player.dart
class MiniPlayer extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: () {
        Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => NowPlayingScreen()),
        );
      },
      child: Container(
        height: 80,
        decoration: BoxDecoration(
          color: Color(0xFF282828),
          boxShadow: [
            BoxShadow(
              color: Colors.black.withOpacity(0.3),
              blurRadius: 10,
              offset: Offset(0, -5),
            ),
          ],
        ),
      ),
    ),
  ),
);

```

```

    ),
    child: Consumer<AudioProvider>(
      builder: (context, provider, child) {
        final song = provider.currentSong;

        if (song == null) return SizedBox.shrink();

        return Column(
          children: [
            // Progress indicator
            StreamBuilder<PlaybackState>(
              stream: provider.playbackStateStream,
              builder: (context, snapshot) {
                final progress = snapshot.data?.progress ?? 0.0;
                return LinearProgressIndicator(
                  value: progress,
                  backgroundColor: Colors.grey[800],
                  valueColor: AlwaysStoppedAnimation<Color>
(Color(0xFF1DB954)),
                  minHeight: 2,
                );
              },
            ),

            // Player content
            Expanded(
              child: Padding(
                padding: EdgeInsets.symmetric(horizontal: 16),
                child: Row(
                  children: [
                    // Album art
                    Container(
                      width: 50,
                      height: 50,
                      decoration: BoxDecoration(
                        borderRadius: BorderRadius.circular(4),
                        color: Colors.grey[800],
                      ),
                      child: song.albumArt != null
                        ? ClipRRect(
                            borderRadius: BorderRadius.circular(4),
                            child: Image.file(
                              File(song.albumArt!),
                              fit: BoxFit.cover,
                            ),
                          )
                        : Icon(Icons.music_note, color: Colors.grey),
                    ),

                    SizedBox(width: 12),

                    // Song info
                    Expanded(
                      child: Column(

```



```

        mainAxisAlignment: MainAxisAlignment.center,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            song.title,
            style: TextStyle(
              color: Colors.white,
              fontWeight: FontWeight.w500,
            ),
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
          ),
          Text(
            song.artist,
            style: TextStyle(
              color: Colors.grey,
              fontSize: 12,
            ),
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
          ),
        ],
      ),
    ),
  ),

  // Play/Pause button
  StreamBuilder<bool>(
    stream: provider.playingStream,
    builder: (context, snapshot) {
      final isPlaying = snapshot.data ?? false;
      return IconButton(
        icon: Icon(
          isPlaying ? Icons.pause : Icons.play_arrow,
          color: Colors.white,
          size: 32,
        ),
        onPressed: () => provider.playPause(),
      );
    },
  ),

  // Next button
  IconButton(
    icon: Icon(Icons.skip_next, color: Colors.white),
    onPressed: () => provider.next(),
  ),
],
),
),
),
),
],
);
},
),

```

```

    ),
  );
}
}

```

## 5. Player Controls Widget:

```

// lib/widgets/player_controls.dart
class PlayerControls extends StatelessWidget {
  final AudioProvider provider;

  const PlayerControls({required this.provider});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        // Secondary controls (shuffle, repeat)
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            IconButton(
              icon: Icon(
                Icons.shuffle,
                color: provider.isShuffleEnabled
                  ? Color(0xFF1DB954)
                  : Colors.grey,
              ),
              onPressed: () => provider.toggleShuffle(),
            ),
            SizedBox(width: 40),
            _buildRepeatButton(),
          ],
        ),
        SizedBox(height: 20),

        // Main controls
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            IconButton(
              icon: Icon(Icons.skip_previous, color: Colors.white, size: 40),
              onPressed: () => provider.previous(),
            ),

            StreamBuilder<bool>(
              stream: provider.playingStream,
              builder: (context, snapshot) {
                final isPlaying = snapshot.data ?? false;
                return Container(

```

```

        width: 70,
        height: 70,
        decoration: BoxDecoration(
          shape: BoxShape.circle,
          color: Color(0xFF1DB954),
        ),
        child: IconButton(
          icon: Icon(
            isPlaying ? Icons.pause : Icons.play_arrow,
            color: Colors.white,
            size: 40,
          ),
          onPressed: () => provider.playPause(),
        ),
      );
    },
  ),
),
],
),
],
);
}

Widget _buildRepeatButton() {
  IconData icon;
  Color color;

  switch (provider.loopMode) {
    case LoopMode.off:
      icon = Icons.repeat;
      color = Colors.grey;
      break;
    case LoopMode.all:
      icon = Icons.repeat;
      color = Color(0xFF1DB954);
      break;
    case LoopMode.one:
      icon = Icons.repeat_one;
      color = Color(0xFF1DB954);
      break;
  }

  return IconButton(
    icon: Icon(icon, color: color),
    onPressed: () => provider.toggleRepeat(),
  );
}
}

```

## 6. Progress Bar Widget:

```
// lib/widgets/progress_bar.dart
class ProgressBar extends StatelessWidget {
  final Duration position;
  final Duration duration;
  final Function(Duration) onSeek;

  const ProgressBar({
    required this.position,
    required this.duration,
    required this.onSeek,
  });

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        SliderTheme(
          data: SliderThemeData(
            trackHeight: 3,
            thumbShape: RoundSliderThumbShape(enabledThumbRadius: 6),
            overlayShape: RoundSliderOverlayShape(overlayRadius: 16),
            activeTrackColor: Color(0xFF1DB954),
            inactiveTrackColor: Colors.grey[800],
            thumbColor: Colors.white,
            overlayColor: Color(0xFF1DB954).withOpacity(0.3),
          ),
          child: Slider(
            value: position.inMilliseconds.toDouble(),
            min: 0.0,
            max: duration.inMilliseconds.toDouble(),
            onChanged: (value) {
              onSeek(Duration(milliseconds: value.toInt()));
            },
          ),
        ),
        Padding(
          padding: EdgeInsets.symmetric(horizontal: 24),
          child: Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Text(
                _formatDuration(position),
                style: TextStyle(color: Colors.grey, fontSize: 12),
              ),
              Text(
                _formatDuration(duration),
                style: TextStyle(color: Colors.grey, fontSize: 12),
              ),
            ],
          ),
        ),
      ],
    );
  }
}
```

```

        ],
    ),
),
],
);
}

String _formatDuration(Duration duration) {
    String twoDigits(int n) => n.toString().padLeft(2, '0');
    final minutes = twoDigits(duration.inMinutes.remainder(60));
    final seconds = twoDigits(duration.inSeconds.remainder(60));
    return '$minutes:$seconds';
}
}

```

## Step 7: Required Features

### Core Features:

#### 1. Music Library:

- Display all songs from device
- Sort by: Title, Artist, Album, Date Added
- Search functionality
- Filter by artist/album

#### 2. Audio Playback:

- Play/Pause
- Next/Previous track
- Seek to position
- Volume control
- Playback speed control (optional)

#### 3. Player Controls:

- Shuffle mode
- Repeat modes (off, all, one)
- Progress bar with time indicators
- Background playback

#### 4. Playlist Management:

- Create custom playlists
- Add/Remove songs from playlists
- Edit playlist name
- Delete playlists
- Recently played list

#### 5. User Interface:

- Home screen with all songs
- Now Playing screen (full screen player)
- Mini player (bottom bar)
- Playlists screen
- Settings screen

## 6. Metadata Display:

- Song title
- Artist name
- Album name
- Album artwork
- Song duration

## 7. Persistence:

- Remember last played song
- Save playback position
- Save playlists
- Save shuffle/repeat preferences
- Save volume level

## Step 8: Advanced Features (Bonus)

### 1. Equalizer:

- Preset equalizer settings
- Custom equalizer
- Bass boost
- Virtualizer

### 2. Sleep Timer:

- Set timer to stop playback
- Fade out audio
- Timer presets

### 3. Lyrics Display:

- Show synchronized lyrics (if available)
- Manual lyrics input

### 4. Audio Effects:

- Crossfade between tracks
- Gapless playback
- Normalize audio levels

### 5. Advanced UI:

- Dynamic themes based on album art
- Animations and transitions

- Gesture controls (swipe to change tracks)
- Visualizer

## 6. **Social Features:**

- Share current song
- Export playlists
- Song statistics

## Step 9: Testing Requirements

### **Manual Testing Scenarios:**

#### 1. **Playback Testing:**

- Play various audio formats (mp3, m4a, flac, etc.)
- Pause and resume playback
- Skip to next/previous track
- Seek forward and backward
- Playback continues in background
- Playback survives app restart

#### 2. **Playlist Testing:**

- Create new playlist
- Add songs to playlist
- Remove songs from playlist
- Reorder songs in playlist
- Delete playlist
- Play entire playlist

#### 3. **Shuffle and Repeat:**

- Shuffle plays random songs
- Repeat all loops playlist
- Repeat one loops current song
- Settings persist after app restart

#### 4. **Permission Handling:**

- Request storage permission
- Handle permission denial
- Handle permission revocation
- Show appropriate messages

#### 5. **Edge Cases:**

- Handle corrupted audio files
- Handle empty music library
- Handle very long song names
- Handle missing metadata

- Handle missing album art

### Unit Tests:

```
// test/services/audio_player_service_test.dart
void main() {
  group('AudioPlayerService Tests', () {
    late AudioPlayerService service;

    setUp(() {
      service = AudioPlayerService();
    });

    test('Initial state is not playing', () {
      expect(service.isPlaying, false);
    });

    test('Load audio file successfully', () async {
      // Test with valid audio file path
      await service.loadAudio('assets/audio/sample.mp3');
      expect(service.currentDuration, isNotNull);
    });

    tearDown(() {
      service.dispose();
    });
  });
}
```

## Step 10: Adding Sample Audio Files

### 1. Create assets folder structure:

```
assets/
├── audio/
│   └── sample_songs/
│       ├── song1.mp3
│       ├── song2.mp3
│       └── song3.mp3
```

### 2. Update pubspec.yaml:

```
flutter:
  assets:
    - assets/audio/sample_songs/
    - assets/images/
```



3. **Include attribution for free music:** Create a `MUSIC_CREDITS.md` file listing all songs and their sources.

## Outcome of this Practice

Upon successful completion, you will have:

- **Audio Playback Expertise** with Flutter audio packages
- **File System Management** skills for accessing local files
- **Permission Handling** implementation experience
- **Complex State Management** for audio player states
- **Background Processing** with audio service
- **Custom UI Components** for music player interface
- **Data Persistence** with user preferences
- **Playlist Management** implementation

Expected Features:

- Complete music library access
- Full audio playback controls
- Shuffle and repeat modes
- Custom playlist creation
- Background audio playback
- Now playing screen
- Mini player
- Seek functionality
- Volume control
- Persistent playback state
- Album artwork display
- Search functionality

## Way to Submit

Required 1: On GitHub

### 1. Create a GitHub Repository:

- **Name:** `flutter_music_player_[your_name]`
- **README.md** must include:
  - Project description and features
  - Setup instructions
  - Screenshots of all screens
  - How to add music files for testing
  - Technologies used
  - Music attribution (if using sample files)
  - Known limitations
  - Future improvements

### 2. Repository Structure:

```
|— README.md
|— MUSIC_CREDITS.md (if including sample songs)
|— screenshots/
|— lib/
|— test/
|— assets/
|   |— audio/
|   |— images/
|— pubspec.yaml
```

### 3. Required Screenshots:

- Home screen with song list
- Now Playing screen
- Mini player
- Playlist screen
- Search screen
- Settings screen
- Permission request dialog

## Required 2: Zip Source Code File on E-Learning

### 1. Prepare Your Project:

- Remove `build/`, `.dart_tool/`, `ios/Pods/` folders
- Include `screenshots/` folder
- Include `test/` folder
- **Important:** Include small sample audio files (< 10MB total)
- Or include instructions for testing with user's own music

### 2. Create ZIP File:

- **Name:** `MusicPlayer_[StudentID]_[Name].zip`

### 3. Upload to E-learning:

- Upload ZIP file
- Add comment with:
  - Audio formats tested
  - Device tested on
  - Extra features implemented
  - Challenges faced

## Submission Checklist:

1. All songs load from device
2. Playback controls working (play, pause, next, previous)
3. Seek functionality working
4. Shuffle mode working
5. Repeat modes working (off, all, one)

6. Playlists can be created and managed
7. Background playback working
8. Mini player functional
9. Now Playing screen complete
10. Permissions handled properly
11. Playback state persists
12. No crashes with edge cases
13. README complete with setup instructions
14. Screenshots included
15. No compilation errors
16. Tested on real device (recommended)

**Deadline:** 11:59 PM - January 19nd, 2025

## Grading Criteria

- **Audio Playback Implementation (30%):** All playback controls, background playback
- **UI/UX Implementation (25%):** Design quality, user experience, responsiveness
- **Core Features (20%):** Library access, playlists, search, metadata
- **State Management (15%):** Proper state handling, persistence
- **Code Quality (10%):** Clean code, proper structure, error handling
- **Documentation (5%):** README, code comments, setup instructions

Bonus Points (up to 15%):

- Equalizer implementation
- Sleep timer
- Lyrics display
- Audio visualizer
- Dynamic theming based on album art
- Crossfade between tracks
- Gapless playback
- Share functionality
- Song statistics
- Advanced animations

## Important Notes

Platform Differences:

- **Android:** Uses MediaStore for music access
- **iOS:** Uses Media Library (requires different permissions)
- Test on both platforms if possible

Audio Format Support:

Common formats supported:

- MP3

- M4A/AAC
- WAV
- FLAC
- OGG

### Performance Considerations:

- Load songs efficiently (pagination)
- Cache album artwork
- Optimize metadata extraction
- Handle large music libraries gracefully

### Common Pitfalls to Avoid:

- Not requesting proper permissions
- Not handling permission denial gracefully
- Loading entire library at once (memory issues)
- Not implementing background playback
- Not persisting playback state
- Not handling corrupted audio files
- Not testing on real devices

### Testing Tips:

- Test with various audio file formats
- Test with missing metadata
- Test with missing album artwork
- Test background playback extensively
- Test on real devices (not just emulators)
- Test with large music libraries (100+ songs)

### Resources for Help:

- Course forum for questions
- Flutter audio documentation
- just\_audio package documentation
- Office hours with instructor

**Academic Integrity:** Your implementation must be original. You may reference documentation and tutorials, but copying code from classmates or online sources without understanding is prohibited.

**Note on Sample Music:** If including sample music files, ensure they are properly licensed (Creative Commons, Public Domain, or with permission). Always provide attribution.

Good luck building your music player! 🎵 🎶